Aula 03 – Mais sobre Controladores

Simulações Computacionais para o Ensino de Robótica Inteligente e Aprendizado por Reforço

Leonardo de Souza Gomes
Orientador: Prof. Dr. André Luiz C. Ottoni

Material adaptado do tutorial oficial Webots [1]
Edital PROGRAD nº 27, de 26 de maio de 2026
Universidade Federal de Ouro Preto

2026

Aula 03 – Mais sobre Controladores

Objetivo: Programar um controlador simples de desvio de obstáculos para o robô e-puck.

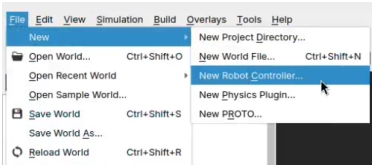
Ao final desta aula, você deverá entender:

- Como inicializar e finalizar corretamente um controlador;
- Como inicializar os dispositivos do robô (sensores e motores);
- Como ler valores de sensores e comandar atuadores;
- Como programar um loop de controle (*feedback loop*) simples.

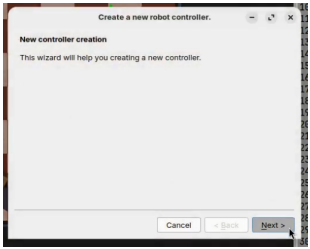
O tutorial em que essa aula se baseia pode ser encontrado em:
https://cyberbotics.com/doc/guide/tutorial-4-more-about-controllers.

Novo Controlador

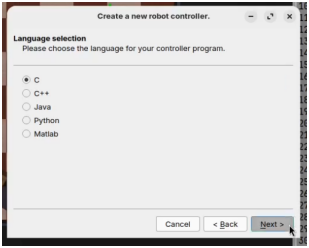
1. Crie um novo controlador em C chamado `epuck_avoid_collision` via File > New > New Robot Controller...;



2. Clique em Next;

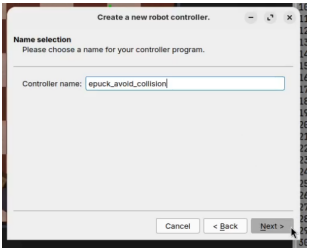


2. Selecione a linguagem de programação C;

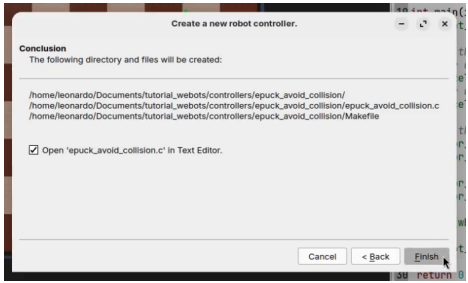


Novo Controlador

3. Nomeie o controller como: `epuck_avoid_collision`;



4. Clique em finish.



Código Base

Substitua o código padrão do arquivo epuck_avoid_collision.c pelo seguinte código:

```c
#include <webots/robot.h>
#include <webots/motor.h>
#define MAX_SPEED 6.28
#define TIME_STEP 64

int main(int argc, char **argv) {
  wb_robot_init();

  WbDeviceTag left_motor  = wb_robot_get_device("left wheel motor");
  WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");

  wb_motor_set_position(left_motor,  INFINITY);
  wb_motor_set_position(right_motor, INFINITY);

  wb_motor_set_velocity(left_motor,  0.8 * MAX_SPEED);
  wb_motor_set_velocity(right_motor, 0.8 * MAX_SPEED);

  while (wb_robot_step(TIME_STEP) != -1);
  wb_robot_cleanup();
  return 0;
}
```

Código Base

Antes:

```c
/*
 * File:          my_controller.c
 * Date:
 * Description:
 * Author:
 * Modifications:
 */

/*
 * You may need to add include files like <webots/distance_sensor.h> or
 * <webots/motor.h>, etc.
 */
#include <webots/robot.h>

/*
 * You may want to add macros here.
 */
#define TIME_STEP 64

/*
 * This is the main program.
 * The arguments of the main function can be specified by the
 * "controllerArgs" field of the Robot node
 */
int main(int argc, char **argv) {
  /* necessary to initialize webots stuff */
  wb_robot_init();

  /*
   * You should declare here WbDeviceTag variables for storing
   * robot devices like this:
```

Depois:

```c
#include <webots/robot.h>
#include <webots/motor.h>
#define MAX_SPEED 6.28
#define TIME_STEP 64

int main(int argc, char **argv) {
  wb_robot_init();

  WbDeviceTag left_motor  = wb_robot_get_device("left wheel motor");
  WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");

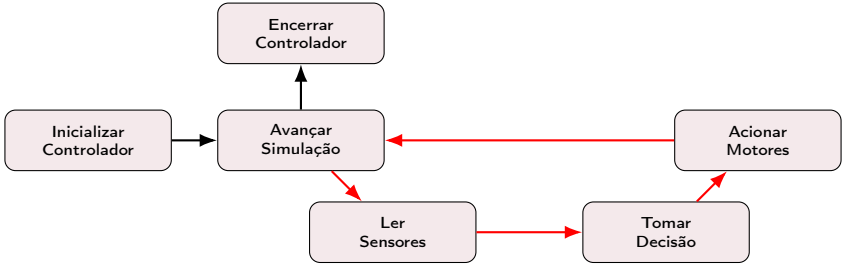
  wb_motor_set_position(left_motor,  INFINITY);
  wb_motor_set_position(right_motor, INFINITY);

  wb_motor_set_velocity(left_motor,  0.8 * MAX_SPEED);
  wb_motor_set_velocity(right_motor, 0.8 * MAX_SPEED);

  while (wb_robot_step(TIME_STEP) != -1);
  wb_robot_cleanup();
  return 0;
}
```

Loop de Controle (Feedback Loop)

Usaremos um loop de controle simples para implementar o comportamento de desvio de obstáculos:



Você programará o robô para seguir em frente até que um obstáculo seja detectado pelos sensores frontais, e então girar em direção ao lado livre.

Entendendo o Modelo do e-puck

A programação do controlador exige conhecer alguns detalhes do modelo do e-puck:

- O robô possui 8 sensores de distância infravermelhos distribuídos ao redor de sua torre, nomeados de `ps0` a `ps7`;
- Cada sensor é modelado por um nó DistanceSensor na hierarquia do robô, acessível via `webots/distance_sensor.h`;
- Os valores retornados variam entre 0 e 4096 (proporcional, de forma linear por partes, à distância);
- 4096 indica muita luz medida (obstáculo próximo); 0 indica ausência de luz (sem obstáculo).

A **API do controlador** é a interface de programação que dá acesso aos sensores e atuadores simulados do robô.

Disposição dos Sensores de Distância

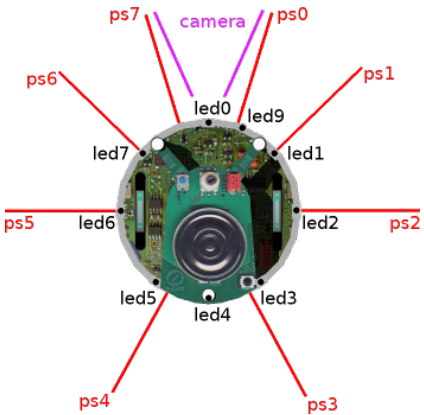


Figura: Vista superior do robô e-puck.

Fonte: Documentação oficial [1].

Detalhes dos sensores infravermelhos:

- A frente do robô está localizada exatamente entre os sensores `ps0` e `ps7`.
- As retas vermelhas indicam a direção de detecção de cada sensor.
- Os rótulos (`ps0` a `ps7`) correspondem aos identificadores oficiais da **API do Webots**.

Programando o Controlador – Includes e Inicialização

Antes de utilizar qualquer sensor do e-puck, é preciso incluir a biblioteca correspondente e inicializar a API do Webots logo no início do programa.

1. No início do arquivo, adicione a linha:
   #include <webots/distance_sensor.h>
2. Logo após a declaração da função `main`, adicione:
   wb_robot_init();

Resultado esperado:

```
#include <webots/robot.h>
#include <webots/motor.h>
#define MAX_SPEED 6.28
#define TIME_STEP 64
// Biblioteca do sensor de distancia
#include <webots/distance_sensor.h>

int main(int argc, char **argv) {
  wb_robot_init(); // Inicializa a API do Webots
  ...
```

Inicializando os Sensores de Distância – Declaração

Com a API inicializada, o próximo passo é declarar os oito sensores de distância do e-puck (ps0 a ps7), que serão referenciados ao longo do controlador.

1. Declare um vetor de `WbDeviceTag` para armazenar os identificadores dos sensores.
2. Declare também um vetor de strings com o nome de cada sensor, conforme definido no mundo do Webots.

Resultado esperado:

```
...
wb_robot_init(); // Inicializa a API

// Inicializacao dos sensores
int i;
WbDeviceTag ps[8];
char ps_names[8][4] = {
  "ps0", "ps1", "ps2", "ps3",
  "ps4", "ps5", "ps6", "ps7"
};
```

Inicializando os Sensores de Distância – Habilitação

Com o vetor de nomes definido, é preciso percorrê-lo para obter cada dispositivo e habilitá-lo com o período de amostragem `TIME_STEP`.

1. Para cada sensor, obtenha o dispositivo pelo nome com wb_robot_get_device().
2. Habilite-o com wb_distance_sensor_enable(), passando o `TIME_STEP` como período de leitura.

Resultado esperado:

```
...
// Inicializacao dos sensores
int i;
WbDeviceTag ps[8];
char ps_names[8][4] = {
  "ps0", "ps1", "ps2", "ps3",
  "ps4", "ps5", "ps6", "ps7"
};
for (i = 0; i < 8; i++) {
  ps[i] = wb_robot_get_device(ps_names[i]);
  wb_distance_sensor_enable(ps[i], TIME_STEP);
}
```

Inicializando os Motores

Com os sensores habilitados, o próximo passo é obter as referências dos motores das rodas e configurá-los para operar em modo de velocidade contínua, começando parados.

1. Obtenha os `WbDeviceTag` dos motores esquerdo e direito pelo nome do dispositivo, usado `wb_robot_get_device(<nome>)`.
2. Defina a posição-alvo como INFINITY para ativar o modo de velocidade contínua.
3. Inicialize a velocidade de ambos os motores como 0.0, deixando o robô parado.

Resultado esperado:

```
...
for (i = 0; i < 8; i++){
  ps[i] = wb_robot_get_device(ps_names[i]);
  wb_distance_sensor_enable(ps[i], TIME_STEP);
}

WbDeviceTag left_motor = wb_robot_get_device("
  left wheel motor");
WbDeviceTag right_motor = wb_robot_get_device("
  right wheel motor");

wb_motor_set_position(left_motor, INFINITY);
wb_motor_set_position(right_motor, INFINITY);

wb_motor_set_velocity(left_motor, 0.0);
wb_motor_set_velocity(right_motor, 0.0);
```

Loop Principal – Lendo os Sensores

Dentro do loop principal (`while`), a cada passo de simulação é necessário ler o valor atual de cada sensor de distância, para que o robô possa reagir ao ambiente.

1. Declare um vetor de `double` para armazenar as leituras dos 8 sensores.
2. Percorra o vetor de sensores `ps[]` e, para cada um, leia o valor atual com `wb_distance_sensor_get_value()`.

Resultado esperado:

```
...
wb_motor_set_velocity(right_motor, 0.0);

// LOOP PRINCIPAL
while (wb_robot_step(TIME_STEP) != -1) {

  // Leitura dos sensores
  double ps_values[8];
  for (i = 0; i < 8; i++)
  ps_values[i] = wb_distance_sensor_get_value(ps
  [i]);
```

Loop Principal – Processando o Comportamento

Com os valores dos sensores já lidos, o próximo passo é interpretá-los para detectar a presença de obstáculos à direita e à esquerda do robô.

1. Verifique se algum dos sensores frontais-direitos (ps0, ps1, ps2) ultrapassa o limiar de 80, indicando obstáculo à direita.
2. Faça o mesmo para os sensores frontais-esquerdos (ps5, ps6, ps7), indicando obstáculo à esquerda.

Resultado esperado:

```
...
// Leitura dos sensores
double ps_values[8];
for (i = 0; i < 8; i++)
ps_values[i] = wb_distance_sensor_get_value(ps[i
  ]);
// Processar comportamento
bool right_obstacle = ps_values[0] > 80 ||
                      ps_values[1] > 80 ||
                      ps_values[2] > 80;

bool left_obstacle = ps_values[5] > 80 ||
                     ps_values[6] > 80 ||
                     ps_values[7] > 80;
```

Loop Principal – Definindo a Velocidade

Com os obstáculos já detectados, o próximo passo é decidir a velocidade de cada roda: por padrão o robô segue em frente, mas desvia ao encontrar um obstáculo de um dos lados.

1. Defina uma velocidade padrão (avançar) para as duas rodas, como metade de `MAX_SPEED`.
2. Se houver obstáculo à direita, gire para a esquerda: inverta o sinal da velocidade esquerda.
3. Senão, se houver obstáculo à esquerda, gire para a direita: inverta o sinal da velocidade direita.

Resultado esperado:

```
...
bool left_obstacle = ps_values[5] > 80 ||
                     ps_values[6] > 80 ||
                     ps_values[7] > 80;

double left_speed  = 0.5 * MAX_SPEED;
double right_speed = 0.5 * MAX_SPEED;

if (right_obstacle) {// Virar para esquerda
  right_speed =  0.5 * MAX_SPEED;
  left_speed  = -0.5 * MAX_SPEED;
} else if (left_obstacle) {// Virar para a direita
  right_speed = -0.5 * MAX_SPEED;
  left_speed  =  0.5 * MAX_SPEED;
}
```

Loop Principal – Enviando aos Atuadores

Por fim, os valores de velocidade calculados são efetivamente aplicados aos motores, fazendo o robô se mover de acordo com o comportamento decidido nesta iteração do loop.

1. Aplique `right_speed` ao motor direito com `wb_motor_set_velocity()`.
2. Aplique `left_speed` ao motor esquerdo da mesma forma.

Resultado esperado:

```
    ...
      left_speed  =  0.5 * MAX_SPEED;
    }

    // Enviar para os atuadores
    wb_motor_set_velocity(right_motor, right_speed);
    wb_motor_set_velocity(left_motor, left_speed);
    };

    wb_robot_cleanup();

  return 0;
}
```

Código Completo do Controlador – Parte 1/4

```c
#include <webots/robot.h>
#include <webots/motor.h>
#include <webots/distance_sensor.h>

#define MAX_SPEED 6.28
#define TIME_STEP 64

int main(int argc, char **argv) {
  wb_robot_init();

  // Inicializacao dos sensores
  int i;
  WbDeviceTag ps[8];
  char ps_names[8][4] = {
    "ps0", "ps1", "ps2", "ps3",
    "ps4", "ps5", "ps6", "ps7"
  };
  for (i = 0; i < 8; i++) {
    ps[i] = wb_robot_get_device(ps_names[i]);
    wb_distance_sensor_enable(ps[i], TIME_STEP);
  }
```

Código Completo do Controlador – Parte 2/4

```
WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");

wb_motor_set_position(left_motor, INFINITY);
wb_motor_set_position(right_motor, INFINITY);

wb_motor_set_velocity(left_motor, 0.0);
wb_motor_set_velocity(right_motor, 0.0);

// LOOP PRINCIPAL
while (wb_robot_step(TIME_STEP) != -1) {

  // Leitura dos sensores
  double ps_values[8];
  for (i = 0; i < 8; i++)
  ps_values[i] = wb_distance_sensor_get_value(ps[i]);
```

Código Completo do Controlador – Parte 3/4

```
// Processar comportamento
bool right_obstacle = ps_values[0] > 80 ||
ps_values[1] > 80 ||
ps_values[2] > 80;

bool left_obstacle = ps_values[5] > 80 ||
ps_values[6] > 80 ||
ps_values[7] > 80;

double left_speed  = 0.5 * MAX_SPEED;
double right_speed = 0.5 * MAX_SPEED;

if (right_obstacle) {
  // Virar para esquerda
  right_speed =  0.5 * MAX_SPEED;
  left_speed  = -0.5 * MAX_SPEED;
} else if (left_obstacle) {
  // Virar para a direita
  right_speed = -0.5 * MAX_SPEED;
  left_speed  =  0.5 * MAX_SPEED;
}
```

Código Completo do Controlador – Parte 4/4

```
    // Enviar para os atuadores
    wb_motor_set_velocity(right_motor, right_speed);
    wb_motor_set_velocity(left_motor, left_speed);
  };

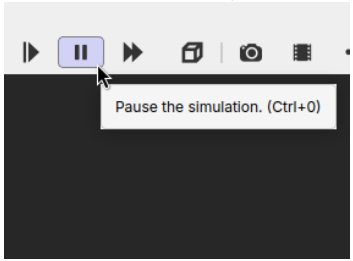
  wb_robot_cleanup();

  return 0;
}
```

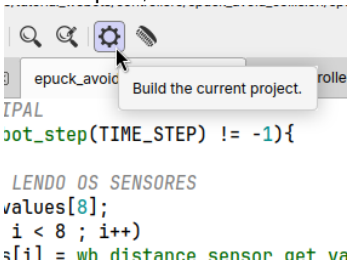
Compilando e Testando o Controlador

Estamos prontos para testar o código! Mas antes, precisamos atribuir esse novo controlador ao robô e-puck. Siga os passos abaixo:

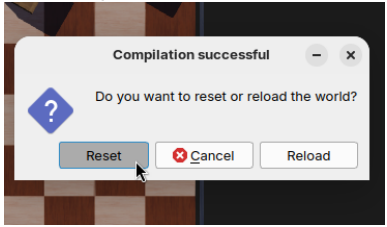
1. Pause a simulação;



2. Clique no botão Build, para compilar;

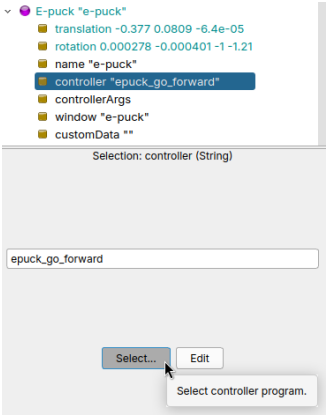


3. Clique Reset;

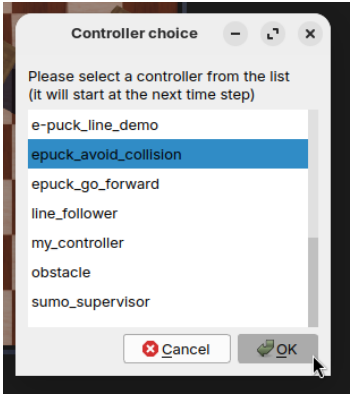


Compilando e Testando o Controlador

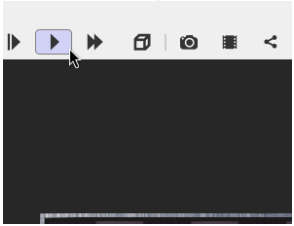
4. No robô e-puck, vá no field controller "epuck_go_foward", depois clique em Select...;



5. Selecione epuck_avoid_collision e clique em OK;



6. Execute novamente a simulação.



Conclusão

Pontos-chave desta aula:

- O ponto de entrada do controlador é a função `main`, como em qualquer programa C padrão;
- Nenhuma função da API do Webots deve ser chamada antes de `wb_robot_init`;
- A última função a ser chamada antes de sair da `main` é `wb_robot_cleanup`;
- Um dispositivo é referenciado pelo campo `name` do seu nó, obtido via `wb_robot_get_device`;
- Cada controlador roda como um processo filho do Webots, sem compartilhar memória com ele, podendo rodar em outro núcleo de CPU;
- O código do controlador é vinculado à biblioteca dinâmica `libController`, responsável pela comunicação entre o controlador e o Webots.

Agradecimentos

Este trabalho foi realizado com o apoio da Pró-Reitoria de Graduação (PROGRAD), do Grupo de Pesquisa Machine Learning and Robotics (MLBots) e do Departamento de Computação (DECOM) da Universidade Federal de Ouro Preto (UFOP).

Leonardo de Souza Gomes, leonardo.sg@aluno.ufop.br

Prof. Dr. André Luiz C. Ottoni, andre.ottoni@ufop.edu.br

Edital PROGRAD nº 27, de 26 de maio de 2026

UFOP
Universidade Federal de Ouro Preto

decom
departamento de computação

ML BOTS
MACHINE LEARNING AND ROBOTICS

Referências

[1] Cyberbotics Ltd., *Tutorial e-puck top view*, Imagem contida em: Webots User Guide - Tutorial 4: More about Controllers. Acesso em: 25 ago. 2026, 2024. endereço: https://cyberbotics.com/doc/guide/tutorial-4-more-about-controllers?tab-language=c.

[2] Cyberbotics, *Tutorial 4: More about Controllers*, https://cyberbotics.com/doc/guide/tutorial-4-more-about-controllers?tab-language=c, Acesso em: 15 ago. 2026, 2026. (acesso em 15/08/2026).